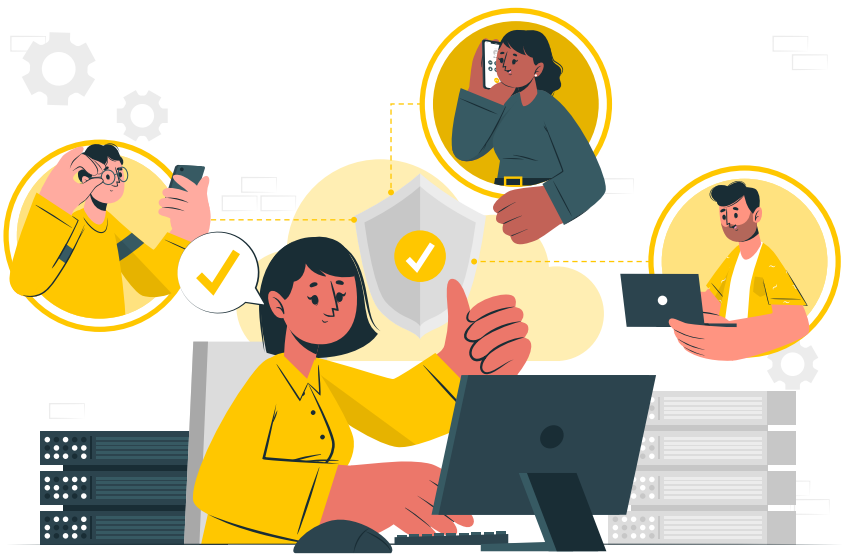
Exploring Behavioural Patterns for **Asynchronous Communication**



A system delivers better throughput when various parts of it work fairly independently without blocking each other. Such a requirement demands asynchronous communication between consumers and providers. This third part of the 'The Design Odyssey' series explores a few of the GoF (Gang of Four) behavioural patterns that help in designing asynchronous communication at the object level. These patterns lay the foundation for popular messaging products like Apache Kafka and many others.

A consumer communicates its request to the provider and the provider, in turn, communicates the response to the consumer. Communication takes place in different forms.

Synchronous communication

In this mode of communication, the consumer sends a request to the provider and waits until it receives the response before proceeding further. In other words, the consumer gets blocked while the provider processes the request.

For example, in the following snippet, a consumer waits until the calculator returns the sum of two numbers.

```
calculator.add(10, 30);
```

Though there are valid cases for this mode of communication, as a side effect, it impacts the overall throughput of the system badly. What if the calculator takes an inordinate amount of time to respond? What if someone else is waiting for a response from the consumer object? As you can see, it can cascade further and the entire system may come to a standstill. For this reason, most modern systems avoid blocking communication between consumers and providers.

Callback

One of the time-tested mechanisms for establishing non-blocking communication between consumers and providers is the use of callbacks. The following snippet captures the essence of such a mechanism:

```
calculator.add(10, 30, sum -> {
     print(sum);
     stop(music);
});
play(music);
```

The objective of the above code is to play some music while the provider computes the sum of the numbers, and to stop the music once the results are printed. As a pattern, the consumer wants to do something (like playing music in this case) while the provider is busy processing the request. This is impossible in synchronous communication.

The third parameter in the *add()* call above is famously known as a *callback* function. A callback function is always executed sometime in the future after the original request is completed. The provider that offers such a mechanism immediately returns the control back to the consumer after receiving the request. The consumer is free to do whatever it